

PatternRecognition – HW3

Objective

The goal of this homework is to understand how to extract radiomics features from medical images and use them to train classical machine learning classifiers for a multi-class image classification task. After feature extraction and selection, you will train and evaluate Random Forest and KNN classifiers on the Chest CT-Scan Images dataset and analyze their performance using standard classification metrics.

Dataset

You will use the publicly available Chest CT-Scan Images dataset hosted on Kaggle:

- <https://www.kaggle.com/datasets/mohamedhanyyy/chest-ctscan-images>

The dataset contains chest CT-scan images organized into four classes: three lung-cancer types (adenocarcinoma, large cell carcinoma, squamous cell carcinoma) and one normal class. The images are already split into train, validation, and test folders.

You must:

- Download the dataset from Kaggle (e.g., using the Kaggle API in Colab).
- Use the provided train / valid / test split as it is.
- Cite the dataset source in your report. Do NOT redistribute the dataset; only the loading code must be in your notebook.

Each image will be treated as a 2D grayscale slice. From each image you will compute a vector of radiomics features (see next section), which will be used as the input to your classical ML classifiers (NOT raw pixels, NOT a CNN).

Preprocessing

- Load the dataset from the Kaggle folder structure (train / valid / test).
- Convert each image to grayscale (single channel).
- Resize images to a fixed size (e.g., 224×224 or 256×256) so feature extraction is consistent.
- Build a binary mask for each image (e.g., simple Otsu threshold or a fixed intensity threshold) so radiomics features can be computed on the relevant region. PyRadiomics requires both an image and a mask.
- Encode class labels as integers (e.g., adenocarcinoma = 0, large cell = 1, normal = 2, squamous = 3). Keep the same encoding across train / valid / test.
- Build feature matrices X_{train} , X_{valid} , X_{test} (rows = images, columns = radiomics features) and label vectors y_{train} , y_{valid} , y_{test} .

Radiomics Feature Extraction

You must extract radiomics features from each image using a radiomics library. The recommended library is PyRadiomics (pip install pyradiomics). It computes hand-crafted features grouped into several families:

- First-order statistics (mean, variance, skewness, kurtosis, energy, entropy, ...).
- Shape-based features (2D shape descriptors of the mask region).
- GLCM – Gray Level Co-occurrence Matrix (texture).
- GLRLM – Gray Level Run Length Matrix (texture).
- GLSZM – Gray Level Size Zone Matrix (texture).
- NGTDM – Neighbouring Gray Tone Difference Matrix (texture).
- GLDM – Gray Level Dependence Matrix (texture).

For each image, the extracted features form one row in your feature matrix. After extraction you will typically have on the order of ~100 numerical features per image. Save the resulting feature matrix as a CSV (or .npy) so feature extraction does not need to be repeated.

Note: PyRadiomics expects SimpleITK images. You will need to convert each NumPy image and mask to a SimpleITK image (`sitk.GetImageFromArray`) before passing them to the extractor.

Feature Selection

Radiomics extraction produces many features, several of which are redundant or weakly informative. You **MUST** apply feature selection on the **TRAINING SET ONLY**, then apply the same selection (and scaler) to the validation and test sets.

Apply at least the following steps in order:

- Remove constant / near-constant features (`VarianceThreshold`) and obvious non-numeric / metadata columns returned by PyRadiomics.
- Standardize features with `StandardScaler` (fit on train only). KNN in particular is very sensitive to feature scale.
- Remove highly correlated features (e.g., drop one feature from each pair with $|\text{Pearson } r| > 0.95$).
- Apply a supervised feature selector. Choose ONE of: `SelectKBest` with `f_classif` or `mutual_info_classif` (e.g., top 20–30 features), OR Recursive Feature Elimination (RFE) with a Random Forest estimator, OR L1-based selection (`LassoCV` / `LogisticRegression` with `penalty="l1"`).

Report how many features remain after each step and which features were selected as the final input to your classifiers.

Classifiers (scikit-learn)

You must train and compare **TWO** classical classifiers from scikit-learn on the selected radiomics features:

- Random Forest – `sklearn.ensemble.RandomForestClassifier`
- K-Nearest Neighbors (KNN) – `sklearn.neighbors.KNeighborsClassifier`

For each classifier:

- Tune at least 2 hyperparameters using cross-validation on the training set (e.g., `n_estimators` and `max_depth` for Random Forest; `n_neighbors` and `weights` for KNN). `GridSearchCV` or `RandomizedSearchCV` is acceptable.
- Refit the best model on the full training set.

- Evaluate the final model on the held-out test set.

The same selected feature set must be used for both classifiers, so their results are directly comparable.

Starter Code Skeleton (PyRadiomics + scikit-learn)

You must complete and adapt the following code skeleton:

```
import os
import numpy as np
import pandas as pd
import SimpleITK as sitk
from PIL import Image
from skimage.filters import threshold_otsu
from radiomics import featureextractor

from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.feature_selection import VarianceThreshold, SelectKBest, f_classif
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import (
    accuracy_score, f1_score, roc_auc_score,
    confusion_matrix, classification_report,
)

# 1. Configure the radiomics extractor (2D mode)
extractor = featureextractor.RadiomicsFeatureExtractor()
extractor.settings["force2D"] = True
extractor.enableAllFeatures()

# 2. Helper: extract features from one image file
def extract_features(img_path, image_size=224):
    img = Image.open(img_path).convert("L").resize((image_size, image_size))
    arr = np.array(img).astype(np.float32)

    # simple mask via Otsu threshold (replace with a better mask if you can)
    thr = threshold_otsu(arr)
    mask = (arr > thr).astype(np.uint8)
    if mask.sum() < 50:          # fallback: use full image
        mask = np.ones_like(arr, dtype=np.uint8)

    sitk_img = sitk.GetImageFromArray(arr[np.newaxis, :, :])  # 1 x H x W
    sitk_mask = sitk.GetImageFromArray(mask[np.newaxis, :, :])

    result = extractor.execute(sitk_img, sitk_mask)
    feats = {k: v for k, v in result.items()
              if not k.startswith("diagnostics_")}
    return feats

# 3. Loop over the train / valid / test folders and build a DataFrame
# DATA_DIR/<split>/<class_name>/<image_file>
```

```

def build_dataframe(split_dir):
    rows = []
    for class_name in sorted(os.listdir(split_dir)):
        class_dir = os.path.join(split_dir, class_name)
        if not os.path.isdir(class_dir): continue
        for fname in os.listdir(class_dir):
            fpath = os.path.join(class_dir, fname)
            try:
                feats = extract_features(fpath)
                feats["label"] = class_name
                rows.append(feats)
            except Exception as e:
                print("skip", fpath, e)
    return pd.DataFrame(rows)

train_df = build_dataframe("Data/train")
valid_df = build_dataframe("Data/valid")
test_df = build_dataframe("Data/test")

# 4. Split features / labels and encode labels
le = LabelEncoder().fit(train_df["label"])
def split_xy(df):
    y = le.transform(df["label"])
    X = df.drop(columns=["label"]).apply(pd.to_numeric, errors="coerce")
    return X.fillna(0.0), y

X_train, y_train = split_xy(train_df)
X_valid, y_valid = split_xy(valid_df)
X_test, y_test = split_xy(test_df)

# 5. Feature selection pipeline (fit on TRAIN only)
vt = VarianceThreshold(threshold=1e-8).fit(X_train)
X_train_v = pd.DataFrame(vt.transform(X_train),
                          columns=X_train.columns[vt.get_support()])

scaler = StandardScaler().fit(X_train_v)
X_train_s = pd.DataFrame(scaler.transform(X_train_v), columns=X_train_v.columns)

# drop highly correlated features
corr = X_train_s.corr().abs()
upper = corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))
to_drop = [c for c in upper.columns if (upper[c] > 0.95).any()]
X_train_s = X_train_s.drop(columns=to_drop)

# univariate selection (or use RFE / L1 instead)
kbest = SelectKBest(score_func=f_classif, k=25).fit(X_train_s, y_train)
selected = X_train_s.columns[kbest.get_support()]
print("Selected features:", list(selected))

def transform(X):
    X = pd.DataFrame(vt.transform(X), columns=X.columns[vt.get_support()])
    X = pd.DataFrame(scaler.transform(X), columns=X.columns)
    X = X.drop(columns=to_drop)
    return X[selected]

```

```

X_train_f = X_train_s[selected]
X_valid_f = transform(X_valid)
X_test_f = transform(X_test)

# 6. Train Random Forest with GridSearchCV
rf_grid = GridSearchCV(
    RandomForestClassifier(random_state=42, class_weight="balanced"),
    param_grid={"n_estimators": [200, 400], "max_depth": [None, 10, 20]},
    cv=5, scoring="f1_macro", n_jobs=-1,
).fit(X_train_f, y_train)
rf = rf_grid.best_estimator_

# 7. Train KNN with GridSearchCV
knn_grid = GridSearchCV(
    KNeighborsClassifier(),
    param_grid={"n_neighbors": [3, 5, 7, 9, 11], "weights": ["uniform", "distance"]},
    cv=5, scoring="f1_macro", n_jobs=-1,
).fit(X_train_f, y_train)
knn = knn_grid.best_estimator_

# 8. Evaluate both classifiers on the test set
def evaluate(name, model, X, y):
    y_pred = model.predict(X)
    y_proba = model.predict_proba(X)
    print(f"\n=== {name} ===")
    print("Accuracy :", accuracy_score(y, y_pred))
    print("F1-macro :", f1_score(y, y_pred, average="macro"))
    print("AUC (OvR):", roc_auc_score(y, y_proba, multi_class="ovr"))
    print(confusion_matrix(y, y_pred))
    print(classification_report(y, y_pred, target_names=le.classes_))

evaluate("Random Forest", rf, X_test_f, y_test)
evaluate("KNN", knn, X_test_f, y_test)

```

Evaluation

For BOTH classifiers (Random Forest and KNN) you must report the following metrics on the test set:

- Accuracy.
- F1-score (macro-averaged AND per-class).
- AUC (multi-class, one-vs-rest, using predict_proba). Report both the macro-averaged AUC and per-class AUC.
- Confusion matrix (rendered as a heatmap with class names on both axes).

Additionally:

- Plot feature importance for the Random Forest (e.g., the top 20 selected features ranked by feature_importances_, as a horizontal bar chart).
- For KNN there is no built-in feature importance. Instead, report permutation importance (sklearn.inspection.permutation_importance) on the validation set, plotted the same way.

- Compare the two classifiers in a small table (Accuracy / F1 / AUC) and discuss which one performs better on this dataset and why.

Report Requirements

- Dataset description (source, number of images per class in train / valid / test).
- Preprocessing pipeline (resizing, grayscale conversion, mask generation).
- Radiomics feature extraction setup (which feature families were enabled, total number of raw features extracted).
- Feature selection pipeline (variance threshold, correlation filter, supervised selector, final number of features and their names).
- Hyperparameter search results for Random Forest and KNN (best parameters, CV score).
- Confusion matrix for each classifier (as a figure).
- Feature importance plot for Random Forest, and permutation importance for KNN.
- Final test-set metrics: Accuracy, F1-score (macro + per-class), AUC (macro + per-class).
- Comparison table: Random Forest vs. KNN.
- Discussion of limitations (mask quality, class imbalance, dataset size) and possible improvements (better segmentation, more advanced selection, ensembles).

Submission

- Colab notebook (.ipynb) with full feature extraction, selection, training, and evaluation code.
- Report (PDF) including all required figures and tables.
- The extracted radiomics feature matrix as a CSV file (so the pipeline is reproducible without re-running PyRadiomics).
- A README that explains how to download the Kaggle dataset and run the notebook.

Late submissions follow course policy.